JSAN CONSULTING · ATLAS OPS

JSAN Vehicle Tracking System Architecture

Driver app, tracking API, admin panel and the coverage / UKM engine: how they are built, hosted and connected.

Document	`docs/ARCHITECTURE.pdf`
Date	13 October 2026
Covers	Backend API, admin panel (web/PWA), Android driver app v1.0.25, Driver Academy, SSDS tool
Screenshots	Captured from the admin panel running locally against the production database, as a superadmin

Contents

1 What the system does
2 System architecture
3 Technology stack
4 Hosting, builds and releases
5 Driver app (Android)
6 Backend API
7 Data model
8 From GPS fix to UKM: the data flow
9 Coverage and UKM rules
10 Roles, access and security
11 Admin panel tour (screenshots)
12 External services
13 Testing
14 Repository layout

1 What the system does

JSAN runs road-survey driving programmes for customers (street-view imagery, HD-map and infrastructure capture) across Europe, Australia, New Zealand and India. This software tracks every survey vehicle, measures how much **new** road each driver covers, and shows managers progress against the customer's road network.

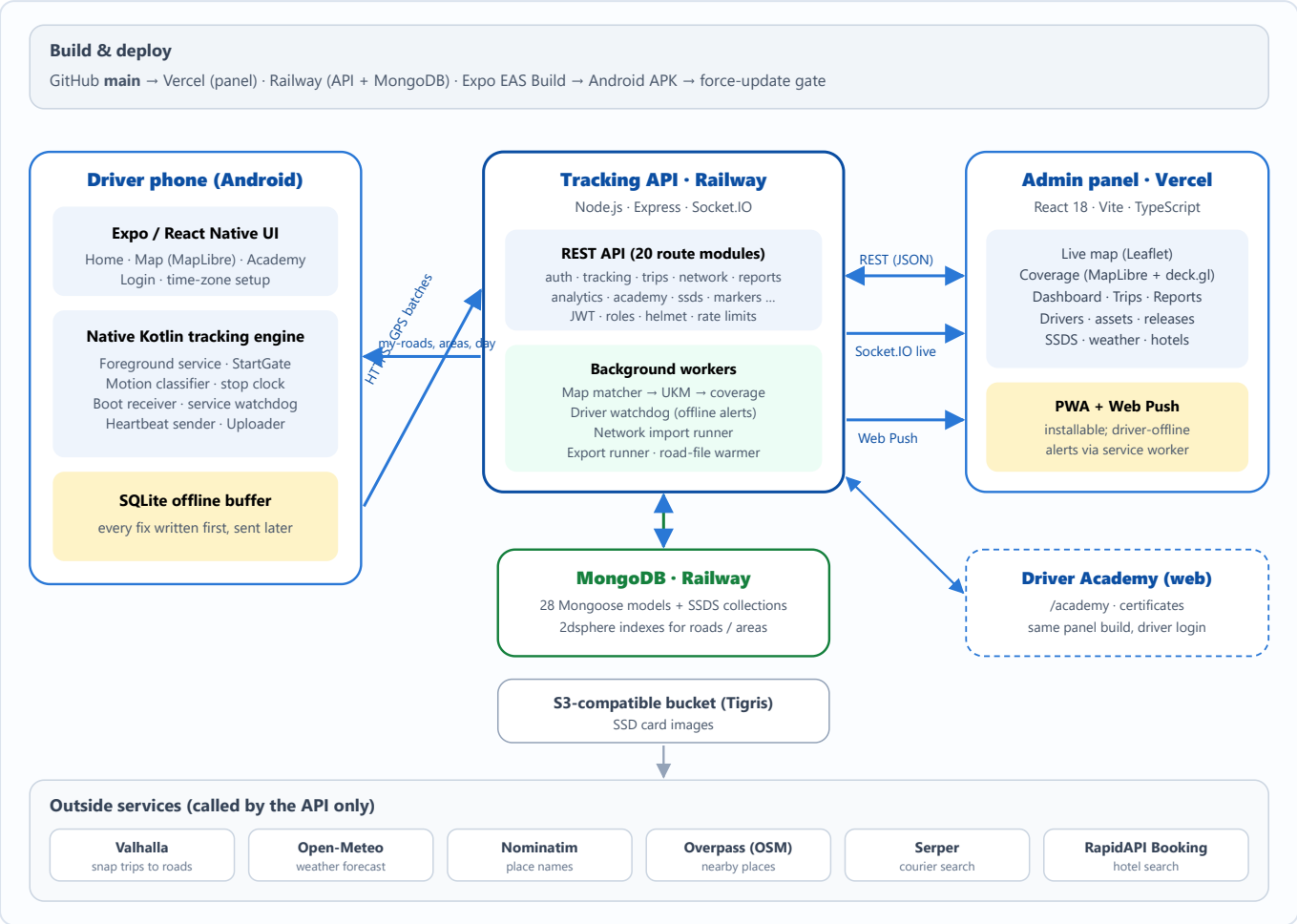
Part	Who uses it	What it does
Driver app	Drivers (Android phones)	Records GPS automatically in the background, works offline, shows the driver their assigned polygons and which roads are still to drive, lets them drop markers, and hosts the Driver Academy course.
Tracking API	The app and the panel	Receives GPS batches, builds trips, snaps them to roads, works out unique km (UKM) and coverage, sends live positions and alerts, imports customer road networks, serves reports.
Admin panel	Superadmins, admins, managers, team leads	Live map, trips, coverage map, dashboard, drivers / vehicles / phones, reports, app health, app releases, SSDS (SSD card) tracking, weather, hotels, couriers. Installable as a PWA with push alerts.
Driver Academy	New drivers	Six-lesson course, in the app and on the web (`/academy`), graded by the server; issues verifiable certificates (`JSAN-XXXX-XXXX`).

Key terms

Term	Meaning
Project	One customer programme (e.g. `PRJ-025-HE-DRIVE-AUSGNZ`). Drivers, vehicles, phones and networks belong to projects.
Trip	One continuous drive. Starts when the phone leaves a 150 m rest circle, ends after the project's stop timeout (default 10 min) at rest.
Network delivery	A customer shapefile import: road links (`RoadLink`) + work-area polygons (`WorkArea`), stored as a `NetworkVersion` , with a region (Australia, New Zealand…).
Polygon / work area	A patch of the network assigned to one driver at a time (`AreaAssignment`), signed off as completed by a manager.
UKM	Unique kilometres: road length a trip was the **first in the whole fleet** to drive. Repeats earn nothing.
Coverage	How much of a delivery's road network has been driven to date (the `LinkCoverage` ledger).

2 System architecture

Three deployable parts, one database. The phone talks only to the API; the panel talks only to the API; the API owns all data and all outside calls.



Map tiles: the panel and the app draw base maps from OpenFreeMap / OpenStreetMap; those requests go straight from the browser or phone to the tile servers.

Design choices that shape everything

- **The phone never loses a fix.** Every GPS point is written to SQLite on the phone first (native code, so it survives the app being killed) and uploaded in batches when there is signal.
- **Raw data is kept; derived data is layered on top.** Raw GPS distance stays forever; road-snapped distance, UKM and coverage are extra fields filled in by background workers and can be recomputed.
- **One UKM number everywhere.** Each trip stores `effectiveUkmMeters` + `ukmBasis`, so the phone, trip page, reports and dashboard all read the same figure.
- **History is append-only where it matters.** Polygon assignments and the coverage ledger keep who-held-what-when, so a trip's numbers do not change when the roster changes.

3 Technology stack

Backend API (`backend/`)

Concern	Technology
Runtime / framework	Node.js, Express 4
Database	MongoDB with Mongoose 8 (geospatial `2dsphere` indexes)
Real-time	Socket.IO 4 (live driver positions, alerts)
Auth	JSON Web Tokens (`jsonwebtoken`), passwords hashed with `bcryptjs`
Hardening	`helmet` , `cors` , `express-rate-limit` , `morgan` request logs
Uploads / files	`multer` (network shapefile zips), `archiver` (zip exports), `xlsx` (Excel import / export)
Geometry	`polygon-clipping` , `delaunator` (area splitting), `tz-lookup` (time zone from coordinates), own geo utils
Notifications	`web-push` (VAPID) to installed panels
Object storage	AWS SDK v3 S3 client → Tigris (S3-compatible) for SSD card images
Tests	`mongodb-memory-server` + `supertest` , 41 suites

Admin panel (`admin-panel/`)

Concern	Technology
UI	React 18, TypeScript, React Router 6, Vite build
Maps	Leaflet + react-leaflet (live map, trips); MapLibre GL 5 + deck.gl 9 (coverage map: every road drawn in WebGL from binary road files)
Real-time	`socket.io-client`
OCR	`tesseract.js` and Google Gemini (reading SSD card numbers and shipping labels in the SSDS tool)
Offline / install	PWA: web manifest + service worker; Web Push alerts

Driver app (`JSAN-AUTO-TRACKING/`)

Concern	Technology
Framework	Expo SDK 54, React Native 0.81, React 19, Expo Router 6
Tracking engine	Custom Expo native module in Kotlin (`modules/…/vehicletracker`): foreground service, Google Play activity recognition, SQLite buffer
Maps	MapLibre React Native 11
Storage / device	`expo-secure-store` (token), `expo-file-system` , `@react-native-community/netinfo` , `expo-location` (permissions)
Builds / updates	EAS Build (preview / production profiles), `expo-updates` , server-side force-update gate
Package	`com.jsan.autotracking` , version 1.0.25

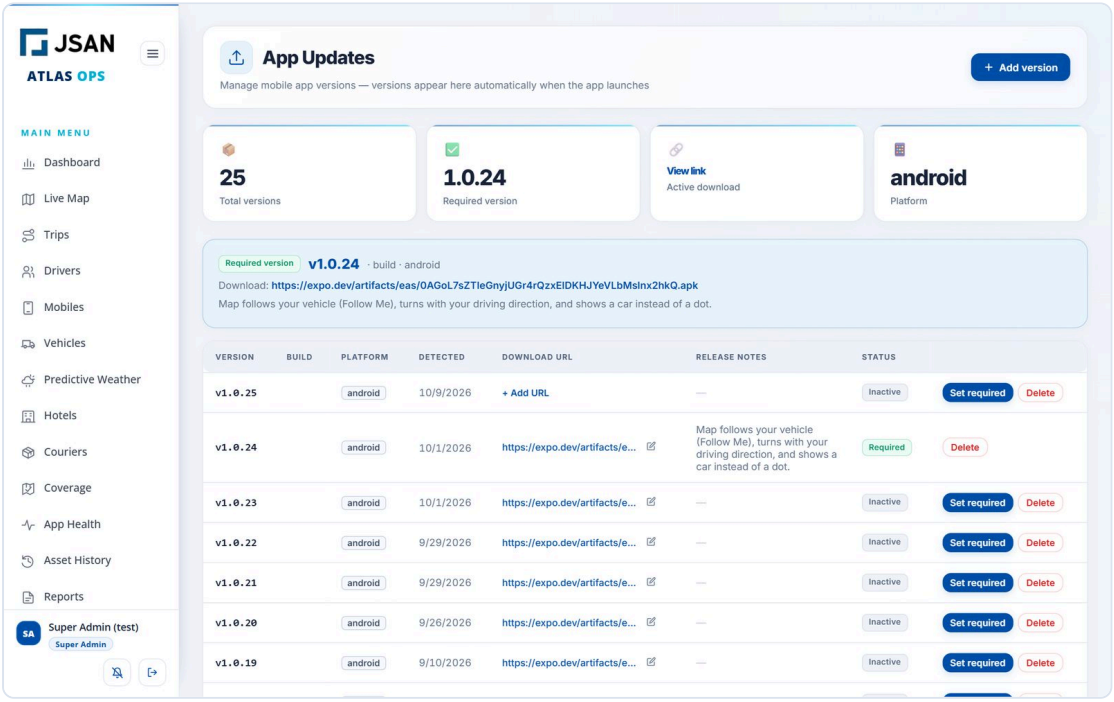
4 Hosting, builds and releases

Part	Host	How it is released
Admin panel	Vercel	Every push to `main` of the outer repository builds (`npm run build` → `dist/`) and goes live. SPA rewrite to `index.html`; the service worker and manifest are served uncached so a new release reaches installed panels.
Tracking API	Railway	Node service on Railway. `GET /health` lists the server's `features` flags — the reliable way to see which code is actually running after a deploy.
Database	Railway MongoDB	One database for tracking and SSDS collections.
Driver app	Expo EAS	EAS Build produces the APK. A release is added on the panel's **App Updates** page; marking it **required** makes older phones show a must-update screen (the force-update gate, `AppVersion` model).
Files	Tigris (S3)	SSD card photos.

> **Release rule.** Pushing `main` deploys the panel to everyone. Work is committed locally and pushed only when the owner says “deploy”. The repository holds two nested git repositories (outer project and `backend/`); commits for both live in the outer one.

Configuration

All secrets live in environment variables on Railway / Vercel / EAS, never in code: `MONGODB_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN`, VAPID keys, S3 keys, third-party API keys, worker intervals (`MAP_MATCH_INTERVAL_SECONDS`, `WATCHDOG_INTERVAL_SECONDS`). In development the panel (`localhost:5173`) proxies to a local API (`localhost:4000`).



App Updates — every Android build, its download link, and which one is required (the force-update gate).

5 Driver app (Android)

A thin React Native shell around a native Kotlin tracking engine. Tracking must keep working with the screen off, the app swiped away, the phone rebooted or offline for hours, so all of it lives in native code; the JavaScript side shows status and maps.

Screens (Expo Router)

Screen	Purpose
Login	Driver signs in; one account may be used on several phones.
Time-zone setup	Confirms the driver's working time zone (days and reports are counted in it).
Home	Tracking state, today's km / UKM / working hours, permission and battery checks, force-update notice.
Map	MapLibre map of the driver's assigned polygons and their roads: blue = driven, red = still to drive; live position; markers.
Academy	The six-lesson course and certificate.

Native tracking engine (Kotlin)

Class	Role
TrackingService	Foreground service that owns location updates and the trip state machine.
StartGate	Trip **start** rule: a trip begins only when the phone leaves a 150 m rest circle — no trips from GPS drift while parked.
MotionClassifier, ActivityTransition / ActivityUpdates receivers	Moving / still decisions using Google Play activity recognition; feeds the anchor-based stop clock that **ends** trips after the project's stop timeout.
LocationDatabase	SQLite offline buffer; every fix is written here first.
Uploader, NetworkUtil, ConnectivityReceiver	Drains the buffer to `POST /api/tracking/ingest` in batches when online; deletes rows only after the server acknowledges them.
HeartbeatSender	Regular “alive” pings; carries per-project settings back to the phone.
BootReceiver, ServiceWatchdog	Restart tracking after a reboot or if Android kills the service.
VehicleTrackerModule, TrackerEvents	The bridge to JavaScript: commands in, live events (position, state) out.

> Detailed reference: `core-tracking-code/how-tracking-works.md` and `JSAN-AUTO-TRACKING/TRACKING.md`.

6 Backend API

Layered Express application: **routes** (URL + role check) → **controllers** (request handling) → **services** (business rules, workers) → **models** (Mongoose). Started by `src/server.js`, which connects to MongoDB, opens HTTP + Socket.IO and starts the workers.

API modules

Mount	Responsibility
/api/auth	Login, logout, current user, permissions
/api/tracking	Phone ingest, my-session / my-areas / my-roads / my-day / my-history; panel live, parked, UKM views
/api/trips	Trip lists, detail, merged daily summaries, exports
/api/network	Shapefile imports, deliveries, work areas, assignment, sign-off, split, coverage reset, road files
/api/analytics	Dashboard: km and UKM by project, region and period
/api/reports	Per-driver daily reports, custody CSV and other exports
/api/users, /projects, /vehicles, /mobiles, /assignments	People, projects and asset custody (who has which vehicle / phone)
/api/markers	Driver-dropped markers and their categories
/api/app, /app-activity, /push	App releases and force-update gate, app health, Web Push subscriptions
/api/academy	Course, server-side grading, certificates and public verification
/api/ssds	SSD card tracking, timesheets, daily status reports
/api/weather, /hotels, /couriers	Field-ops helpers backed by outside services

Background workers (in the API process)

Worker	What it does
Map matcher	Picks up finished trips, snaps them to roads through Valhalla, then runs global UKM and network coverage attribution.
Driver watchdog	Spots drivers mid-trip whose phone stopped reporting; raises one “driver offline” alert (Socket.IO + Web Push), de-duplicated across restarts.
Import runner	Processes uploaded network shapefile zips into roads + polygons (report first, then commit).
Export runner	Builds large exports (KML / JSON / zip) in the background.
Road-file warmer	Prebuilds the binary road files the coverage map draws, for every live delivery.

Real-time channel

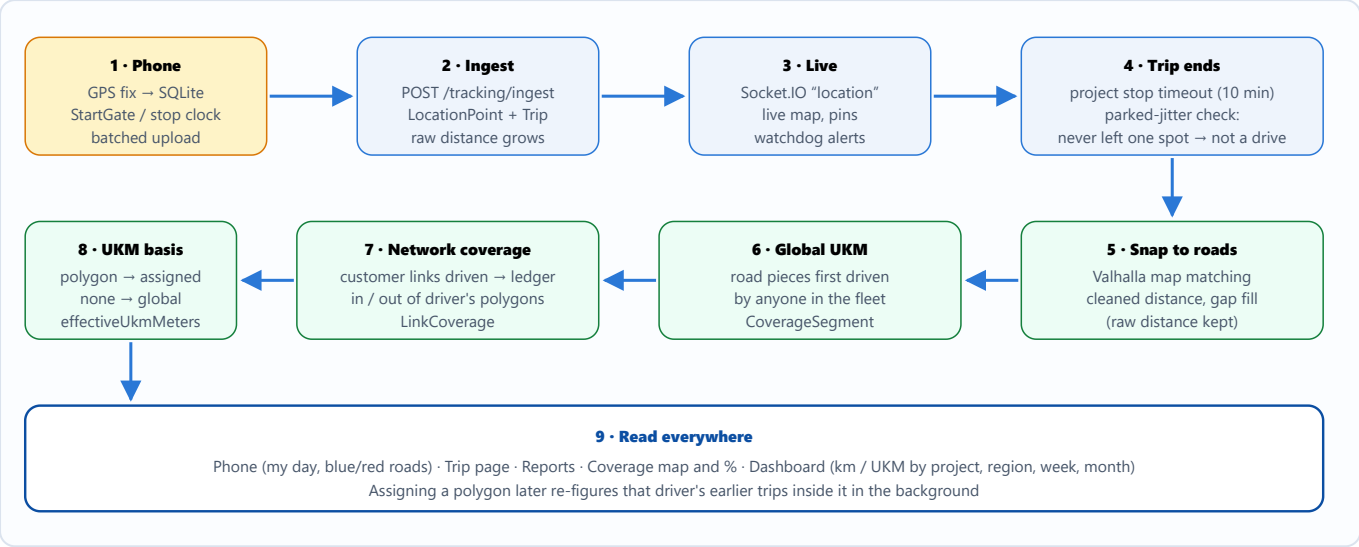
Socket.IO events `location` (each driver's latest position) and `alert`. The panel's live map and coverage pins update from these without polling.

7 Data model

28 Mongoose models in `backend/src/models/`, plus the SSDS tool's collections (drivers, timesheets, daily_reports, …) in the same database.

Group	Models	Notes
People & projects	User, Project	Roles: superadmin, admin, manager, team_lead, user (driver). A driver can be on several projects.
Assets	Vehicle, MobileDevice, Assignment	Custody history: which driver held which vehicle / phone, when. A driver may hold several at once.
Tracking	Trip, LocationPoint, RejectedPoint	Trip carries raw distance, snapped distance, in/out-of-area km, UKM fields and statuses. LocationPoint is the raw GPS stream.
Global UKM	CoverageSegment, UkmEdge	Fleet-wide record of road pieces driven, for UKM where there is no customer network.
Customer network	NetworkVersion, RoadLink, WorkArea, ImportJob	One NetworkVersion per delivery (target length, region). RoadLink = customer road link with geometry. WorkArea = polygon (areaCode is the stable id).
Coverage ledger	LinkCoverage	One row per driven link per delivery: who drove it first, when, length. Source of coverage % and assigned UKM.
Area workflow	AreaAssignment, AreaCompletion, AreaSplit, CoverageReset	Append-only assignment history; manager sign-off per coverage cycle; zones split by km; “clear driven data”.
Field	Marker, MarkerCategory	Flags dropped by drivers (red / yellow / blue categories).
App ops	AppVersion, AppActivity, PushSubscription, ExportJob	Releases and force-update, app health, push targets, background exports.
Field-ops helpers	HotelBooking, HotelLocation, CourierLocation	Hotels and couriers near the crews.

8 From GPS fix to UKM: the data flow



Notes on the flow

- **Offline first:** a phone out of signal for hours uploads its backlog later; the server builds the trip from the timestamps, not the arrival time.
- **Distances:** *km driven* on every screen is the raw GPS distance (`distanceMeters`); the road-snapped figure (`cleanedDistanceMeters`) is kept beside it.
- **Working hours:** each trip's start to end, capped at 12 h per trip; imported trips have no real times and are left out of hours.
- **Phone roads:** the phone only receives roads and outlines for polygons assigned to that driver; driven roads show blue for whoever holds the polygon now.

9 Coverage and UKM rules

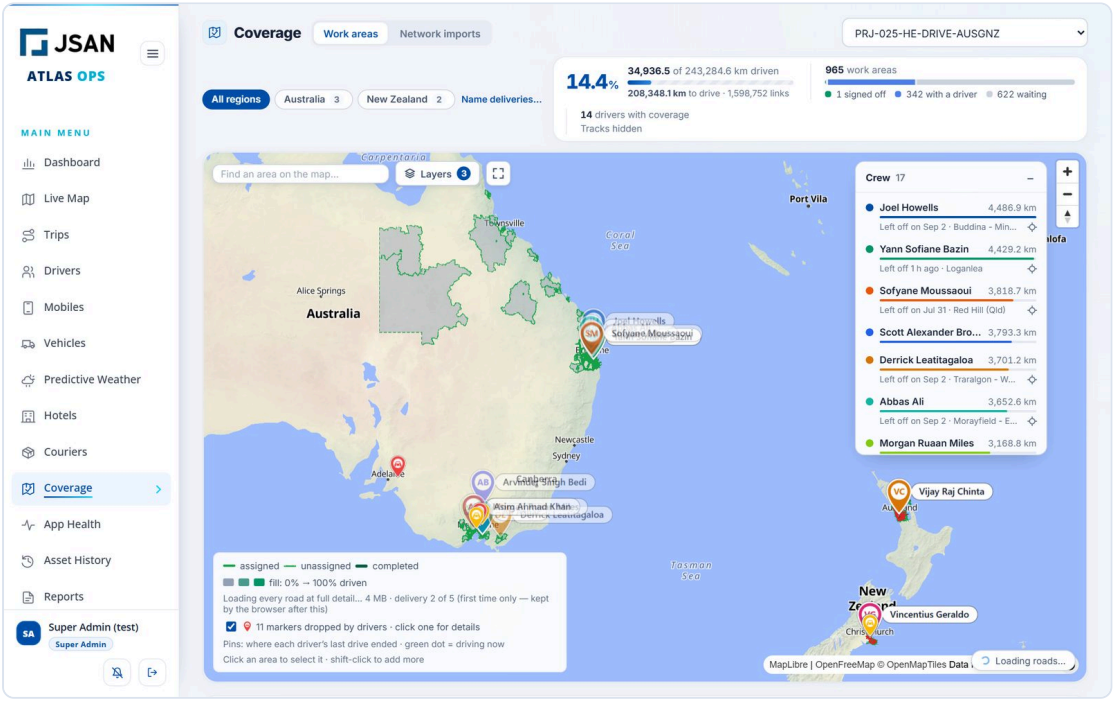
Coverage is fleet-wide: the **first** trip to drive a customer road link owns it. Every later pass by anyone is a repeat and earns no UKM. How a trip's UKM is measured depends on whether its driver holds a polygon:

Trip	Counts as UKM	Shown as
Driver holds polygon(s)	Customer links **inside their polygons** this trip was first to drive (15 m buffer, 60 % of the link, heading check)	Assigned UKM
Polygon assigned *after* the trip	Roads the driver drove there first move into their assigned UKM — same driver, the work was just done early	Assigned UKM
No polygon (free drive)	Any road this trip was first in the fleet to drive	Free-drive UKM
Polygon-holder outside their polygons	New customer links there are credit for the project, **not** the driver's UKM	Outside UKM (separate)

> **UKM = Assigned UKM + Free-drive UKM.** Outside UKM is reported beside it, never added in. Coverage % = driven length of a delivery's links ÷ the delivery's total length, to date.

Area workflow

- **Import** a delivery (polygons + road shapefiles, column mapping remembered per project); several deliveries of different ground can be live at once.
- **Region** per delivery is detected from the time zone under its polygons (Australia, New Zealand…) and can be renamed.
- **Assign** a polygon to one driver; **split** a too-big polygon into zones by km; **sign off** when complete; **clear driven data** to restart an area.

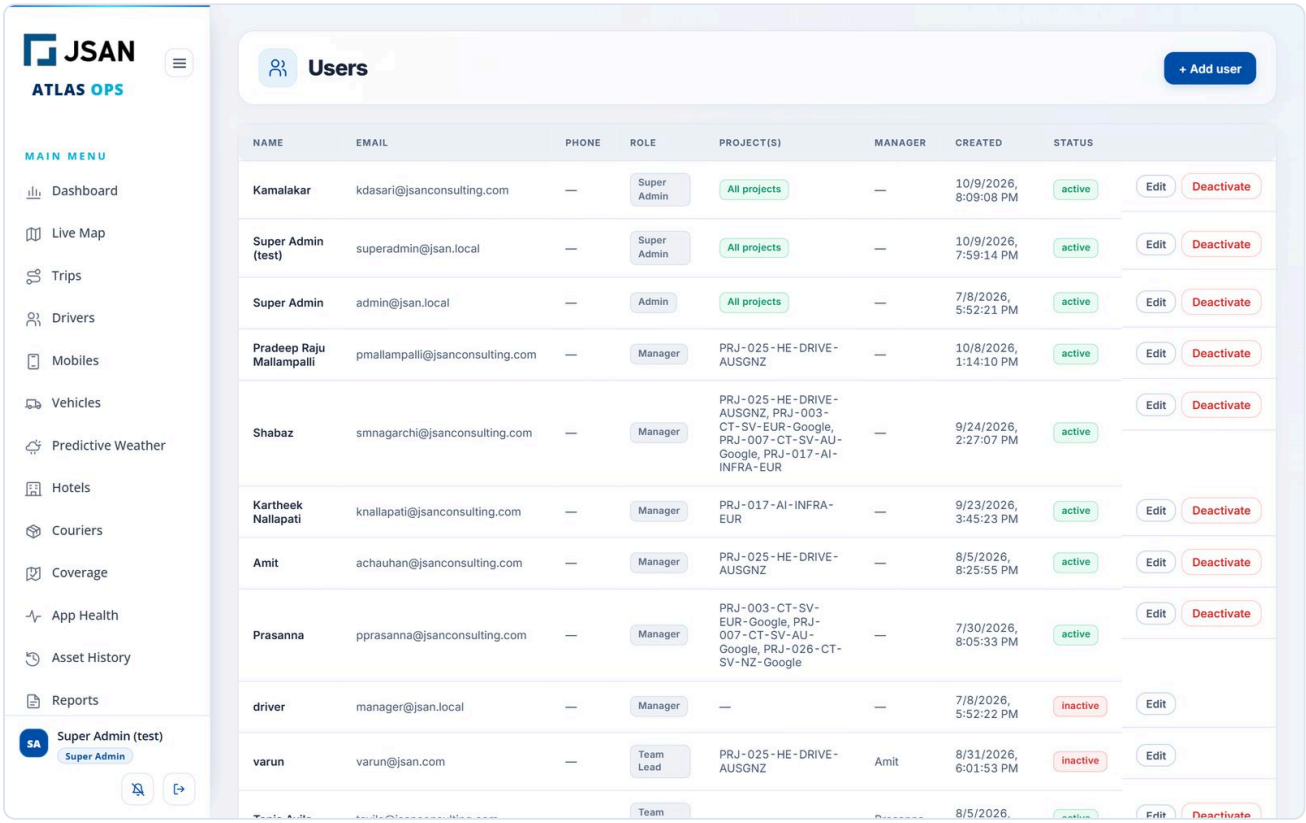


Coverage — PRJ-025's live deliveries in Australia and New Zealand: 14.4 % of 243,285 km driven, 965 work areas, crew list with each driver's km and where they left off.

10 Roles, access and security

Role	Can see / do
Superadmin	Everything, all projects; creates admins and other superadmins.
Admin	Everything, all projects (admins and superadmins are given every project automatically); dashboard, users, projects, app releases, markers setup, SSDS.
Manager	Their projects: live map, trips, coverage, assigning and signing off areas, drivers, reports.
Team lead	Their projects' operational views (live map, trips, coverage, UKM), as set by the role's permissions.
User (driver)	The driver app, their own SSDS pages and the Academy. Only their own data and only roads of polygons assigned to them.

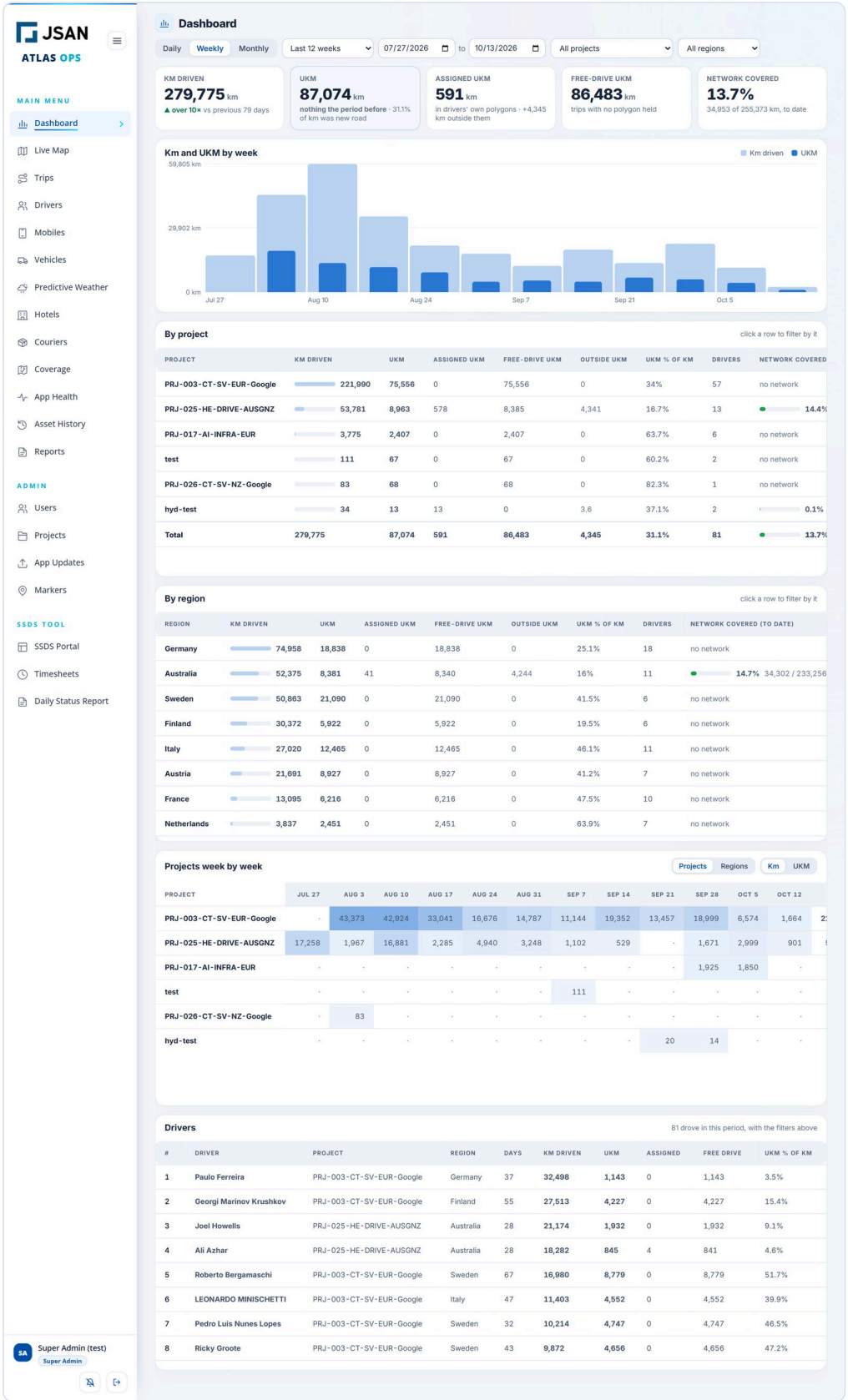
- **Authentication:** email + password (bcrypt hashes) → JWT. Panel sessions last one week. A driver account may sign in on several phones.
- **Authorisation:** every route declares the roles it admits (`requireRole`); project-scoped reads check the caller's projects.
- **Transport and headers:** HTTPS on Railway / Vercel; `helmet` security headers; rate limits on the heavy phone sync endpoints.
- **Secrets:** environment variables only. Keys that have ever been pasted into chats or code should be rotated.
- **Code in the browser:** any web app's JavaScript can be read by its users; protection comes from server-side checks, not from hiding the code.



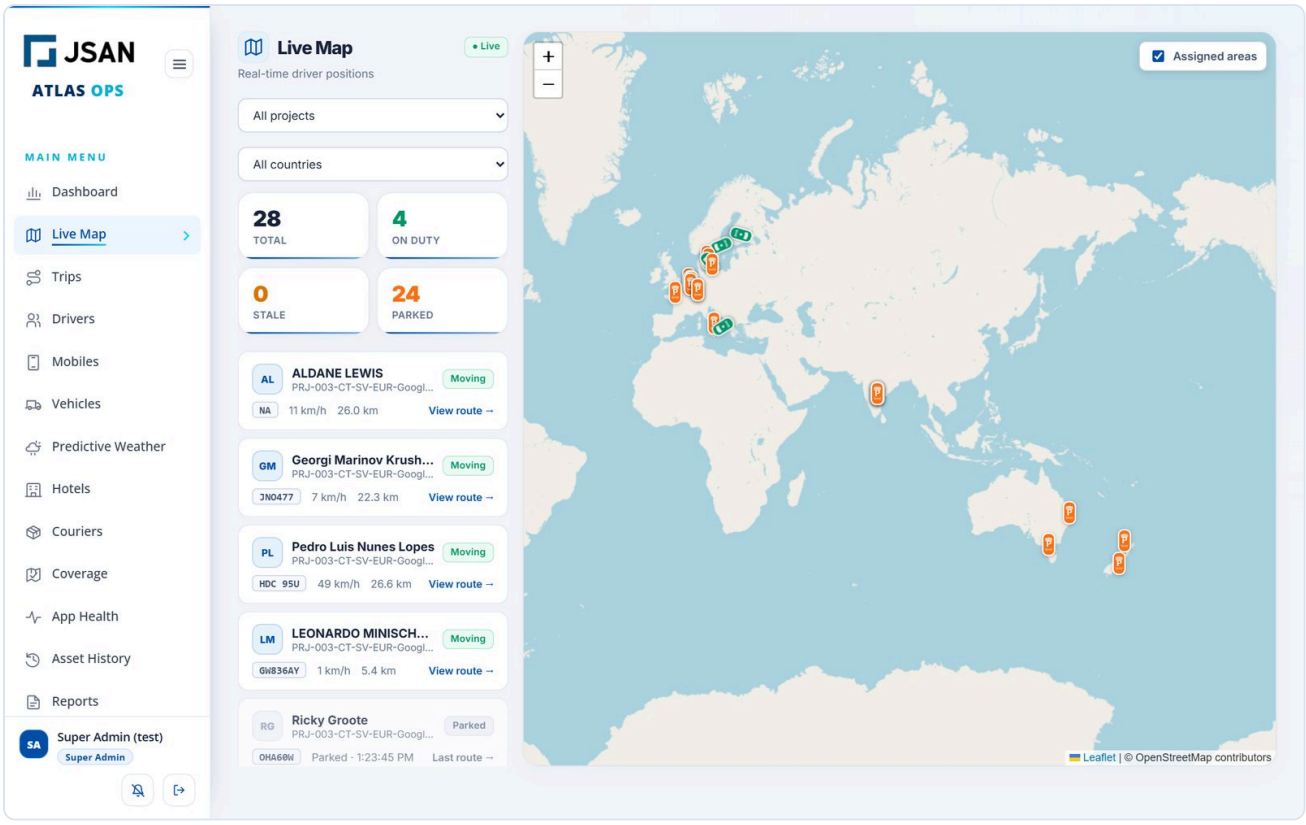
Users — accounts, roles and the projects each one may see.

11 Admin panel tour

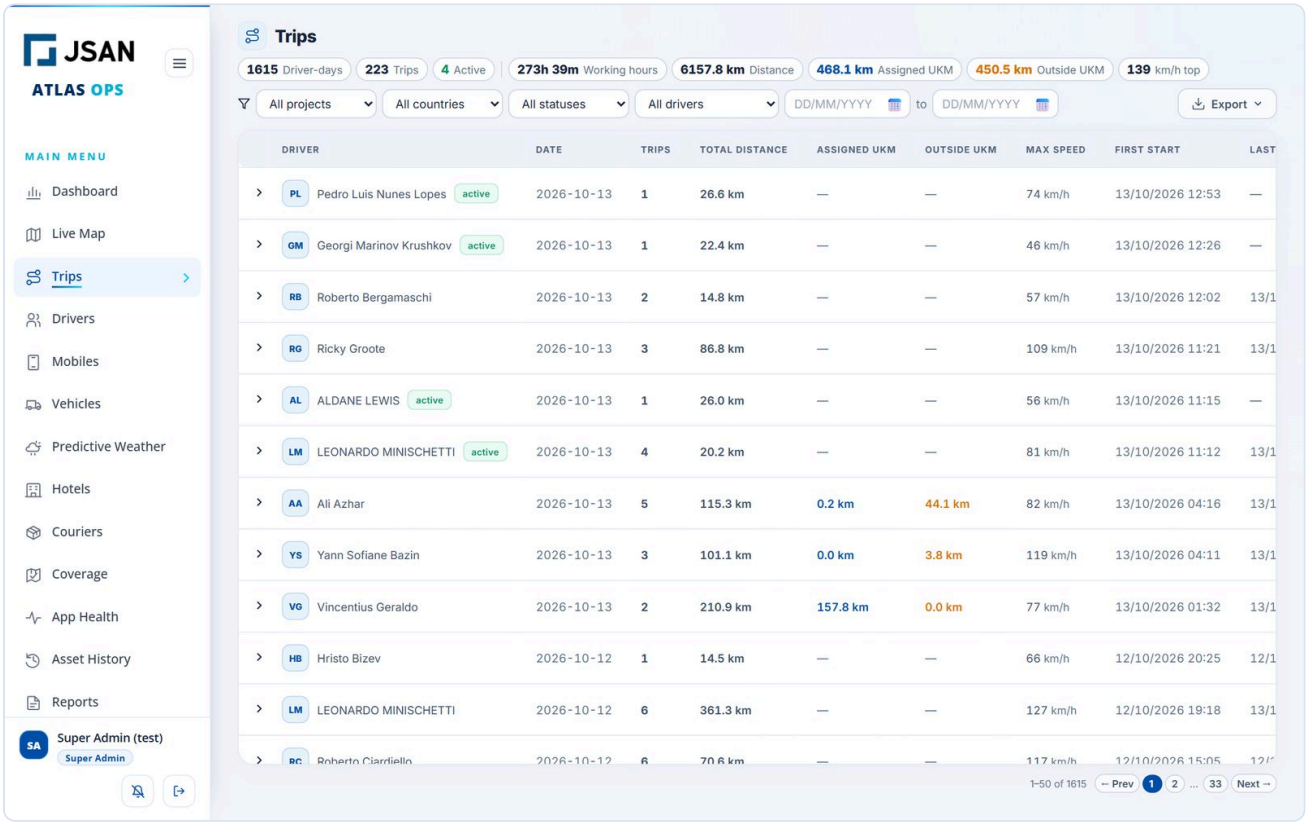
Captured on 13 October 2026 from the panel running locally against the production database, signed in as a superadmin.



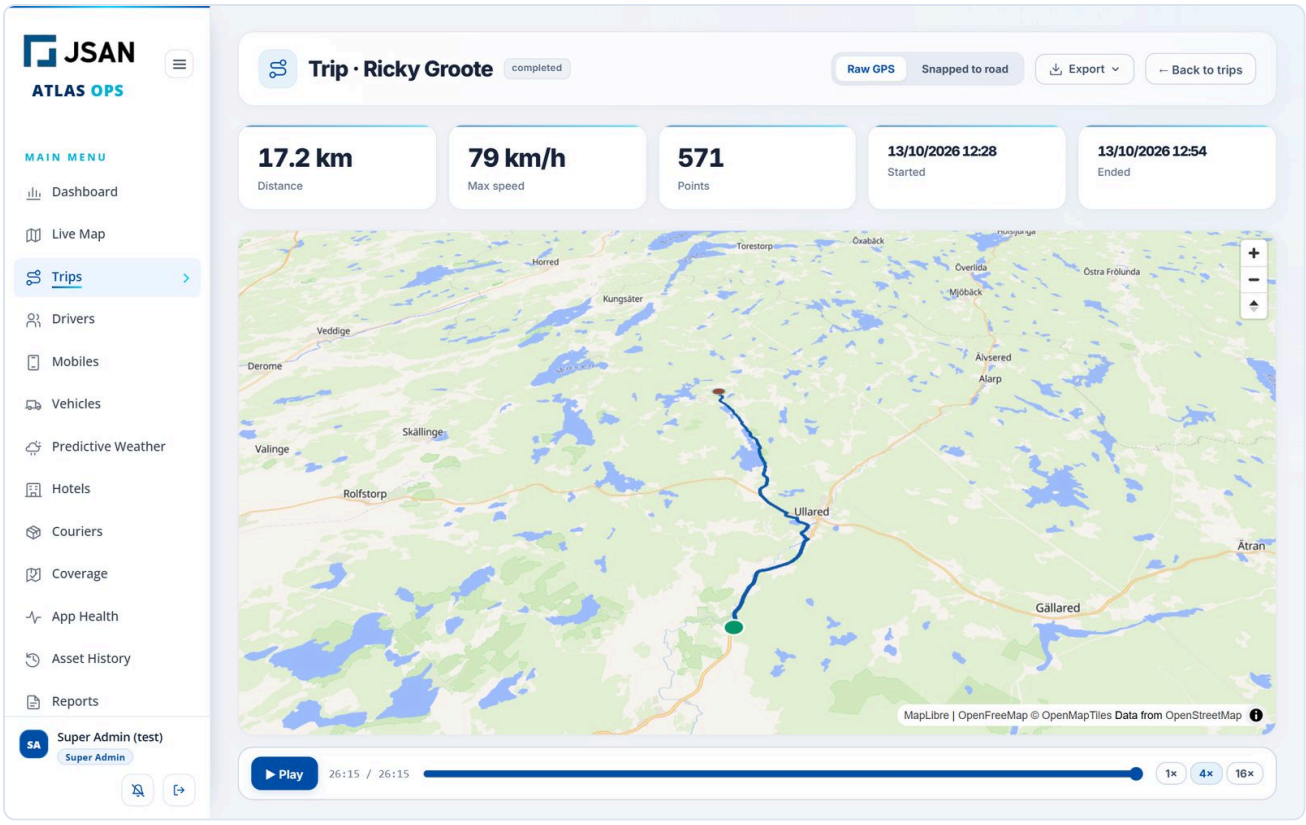
Dashboard — km and UKM only. Daily / weekly / monthly, any period, filtered by project and region. Totals, km-vs-UKM chart, by-project and by-region tables with network covered to date, project-by-week table, drivers.



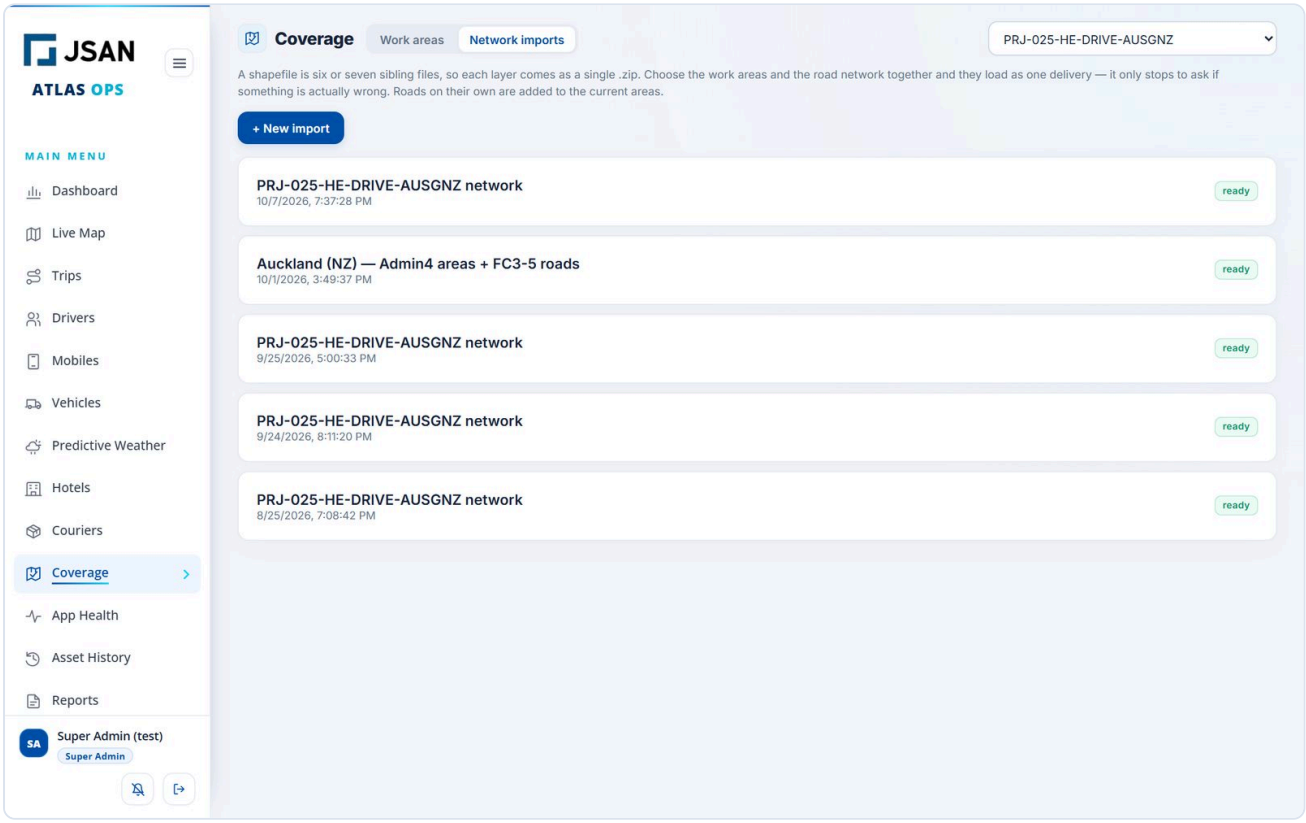
Live Map — every driver's position in real time (Socket.IO), moving / parked / stale counts, project and country filters, assigned areas overlay.



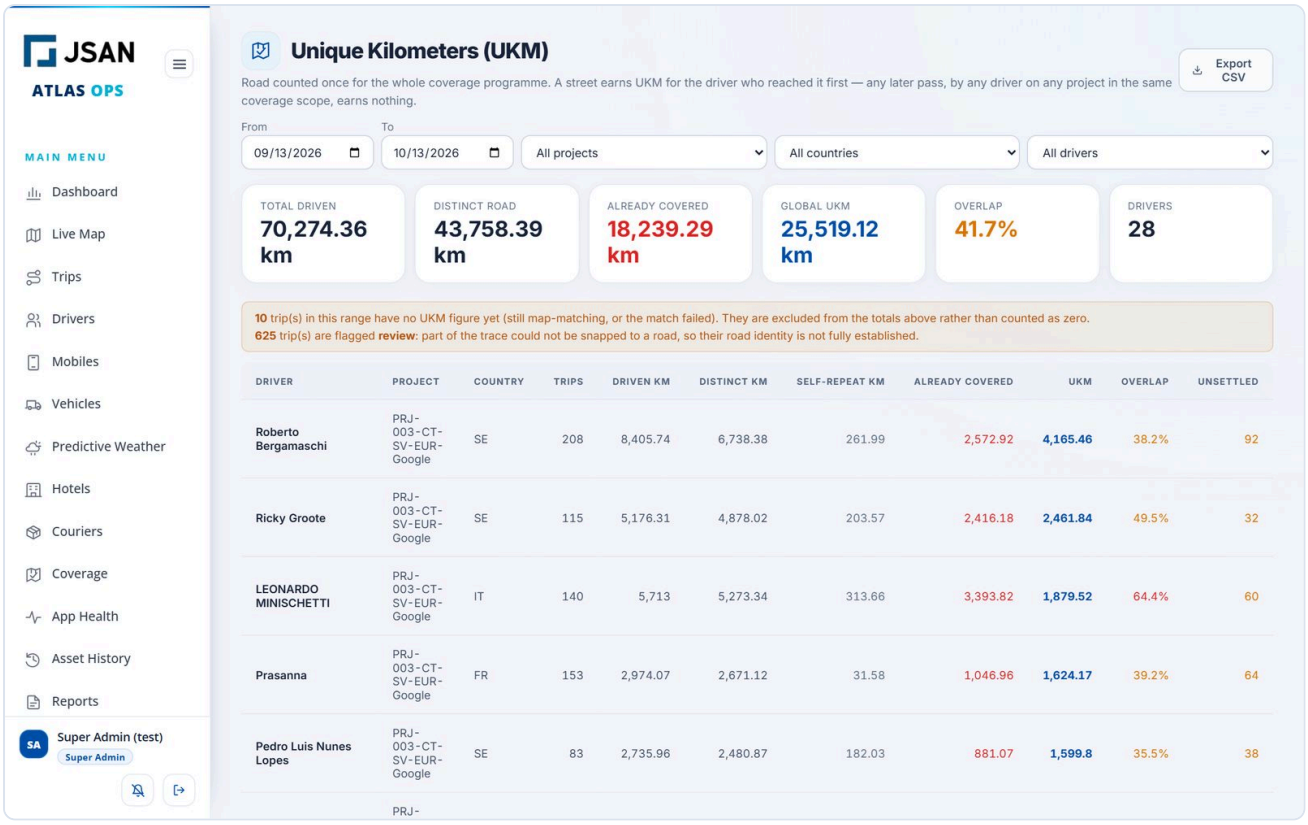
Trips — one row per driver per day with distance, assigned / outside UKM, working hours, speeds; filters and export.



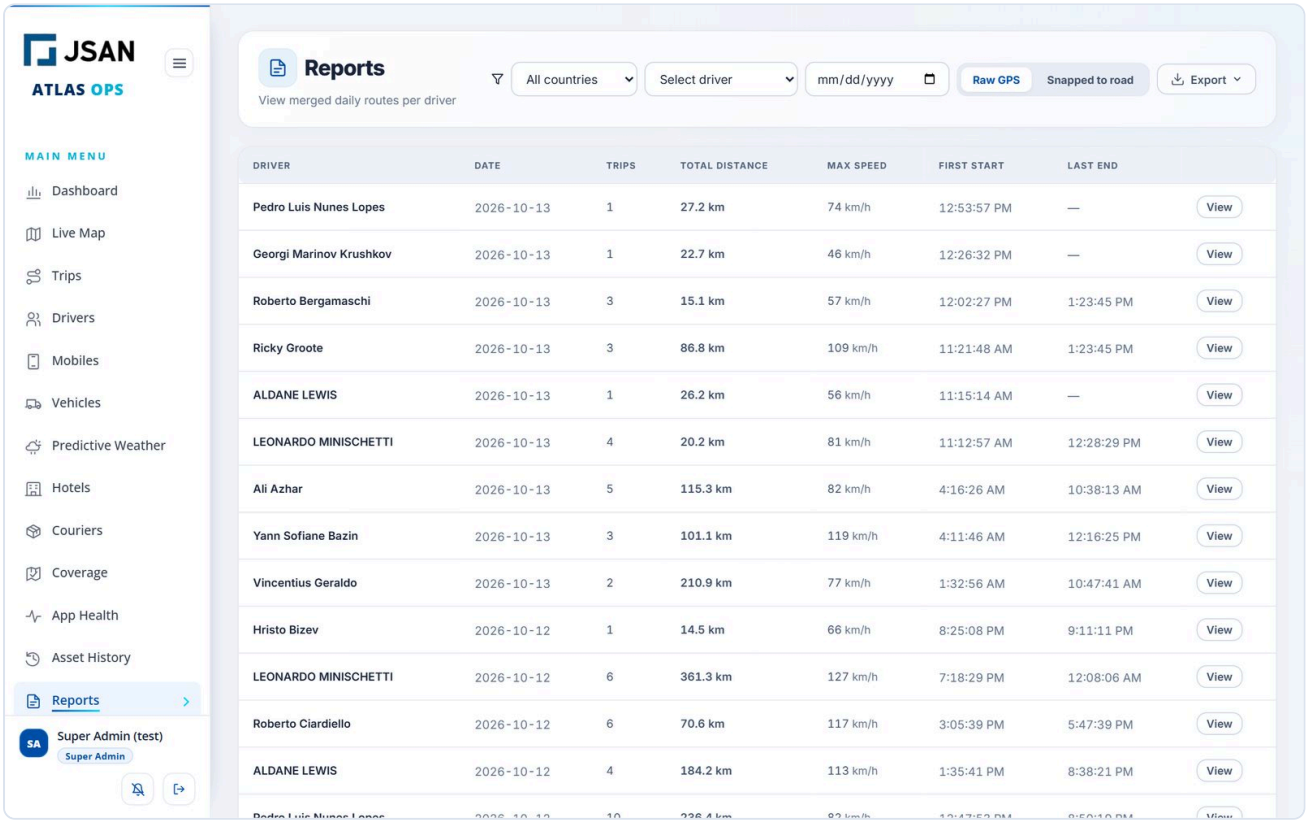
Trip detail — the route on the map, raw GPS or snapped to road, replay with speed controls, KML / JSON export.



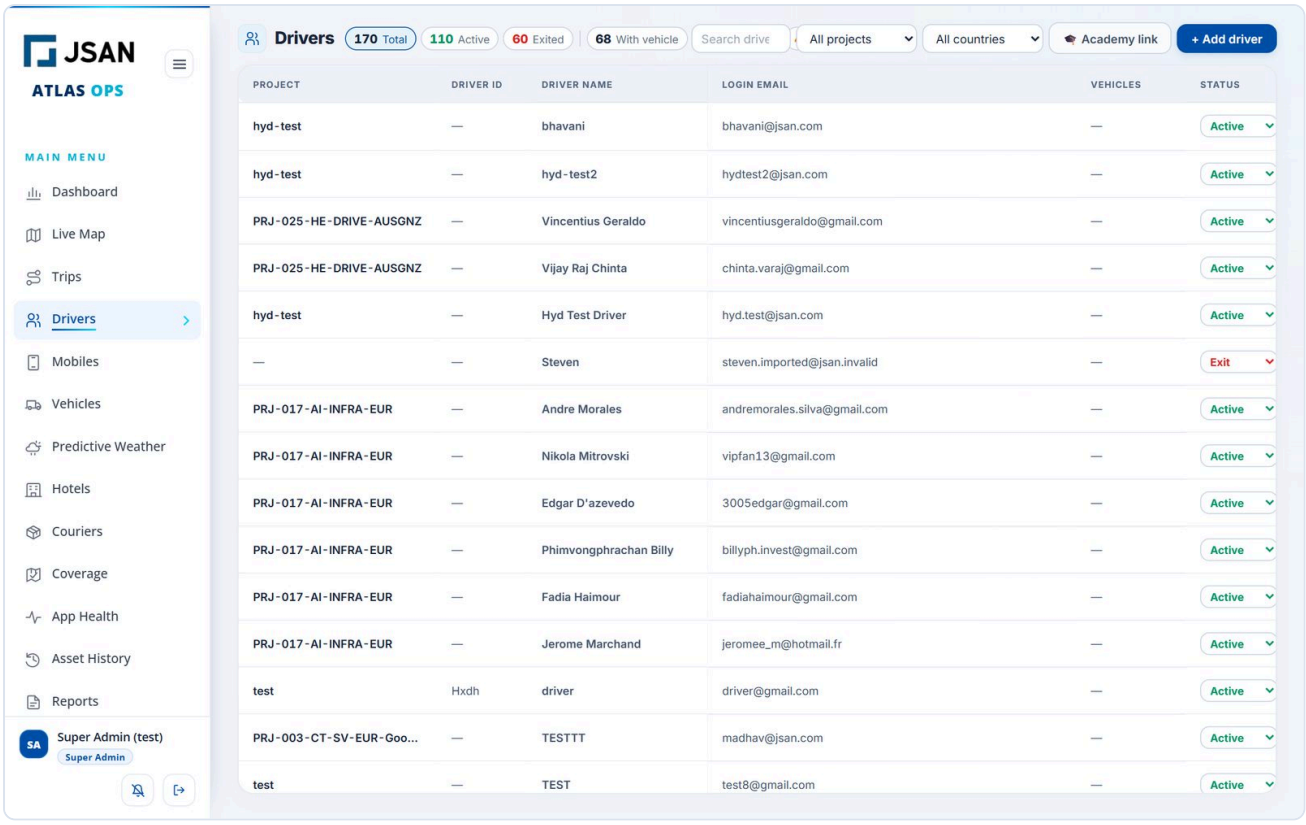
Coverage → Network imports — customer shapefile deliveries, each imported as polygons + road network.



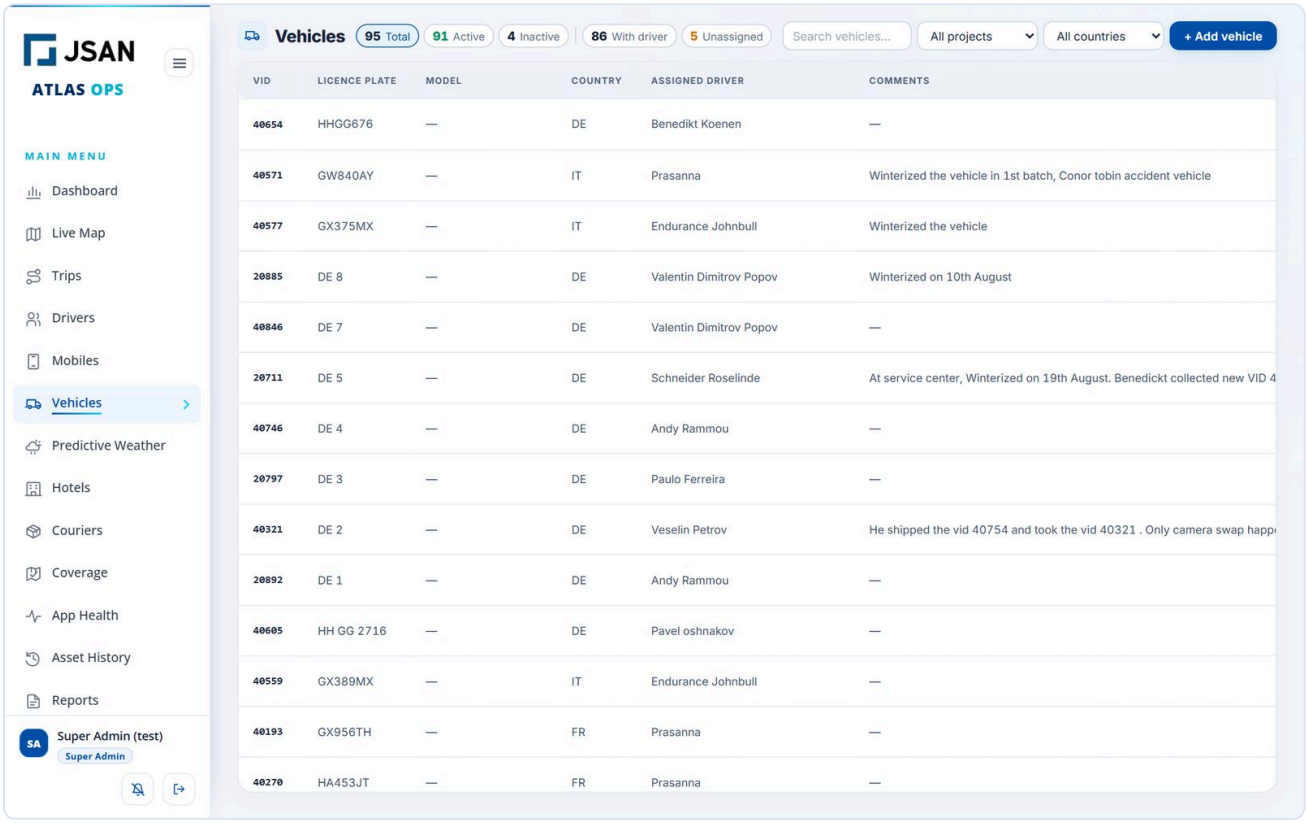
UKM — per-driver unique km for any period: total driven, distinct road, already covered, global UKM, overlap.



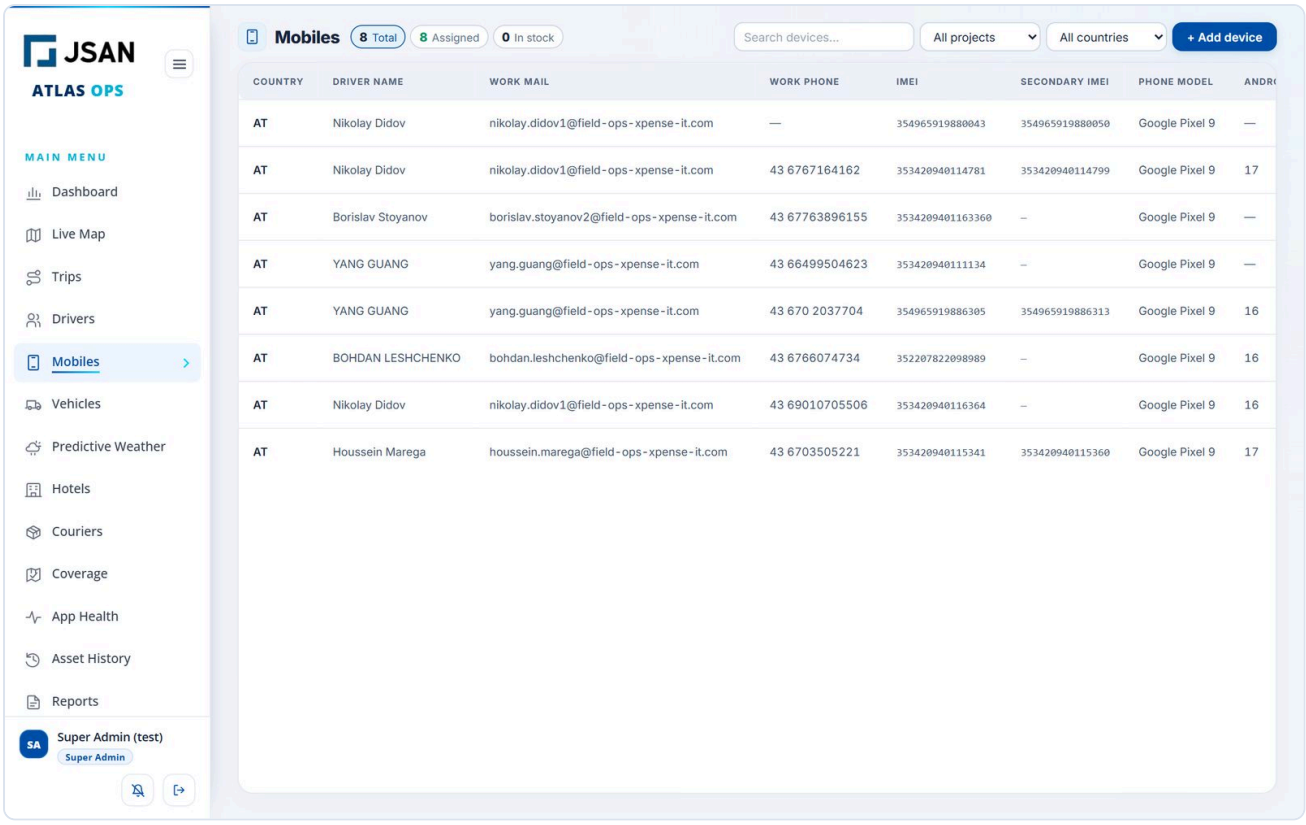
Reports — merged daily routes per driver with distance, speed, first start and last end; export.



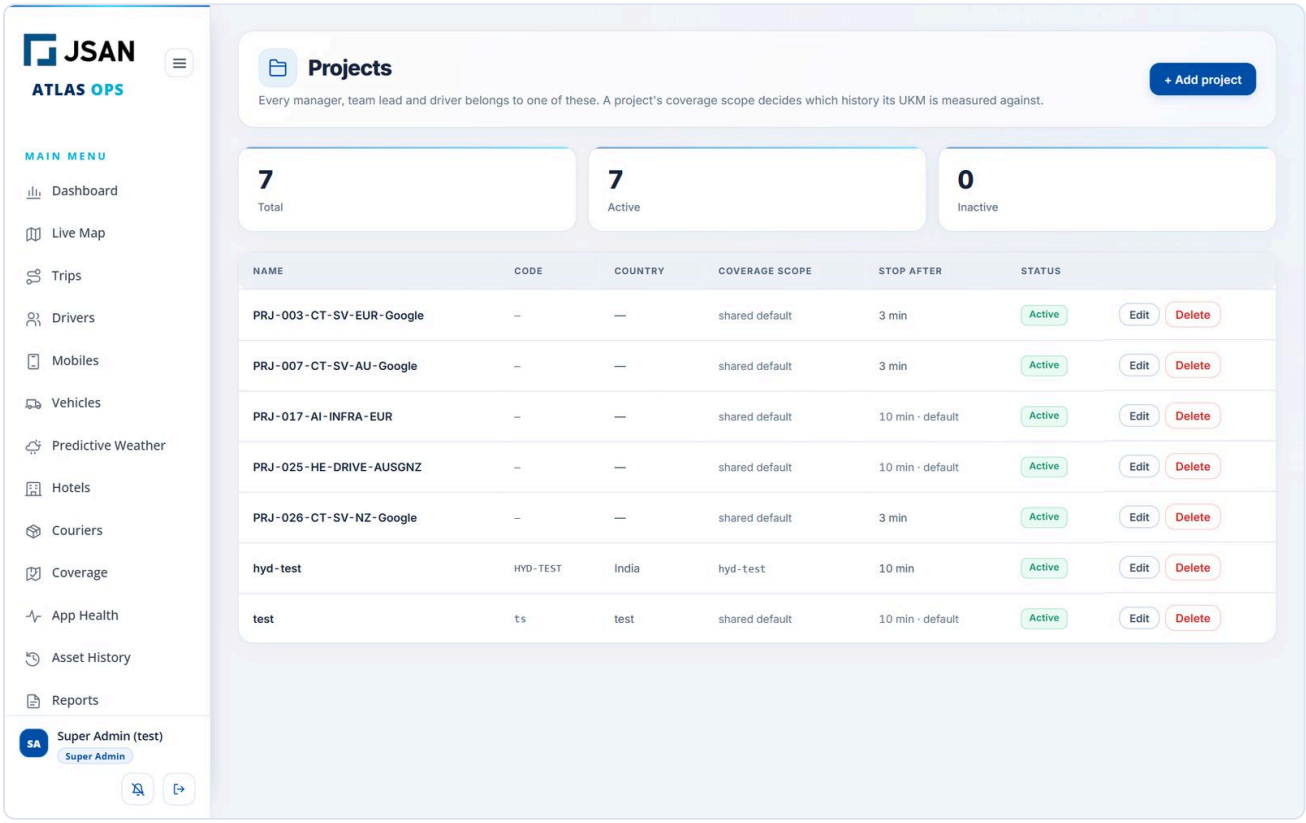
Drivers — roster with project, status, vehicle, login and Academy completion.



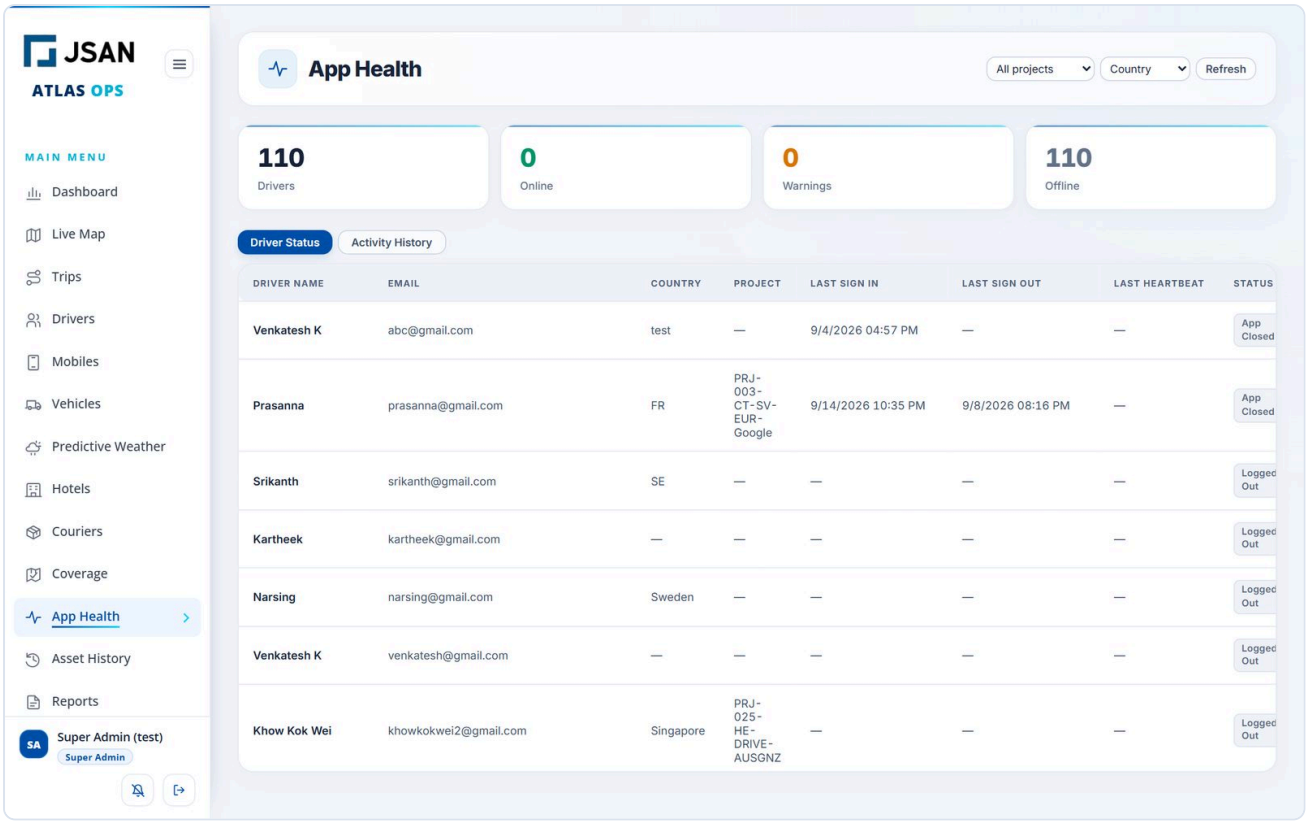
Vehicles — fleet with licence plates, country, assigned driver and notes; custody history is on Asset History.



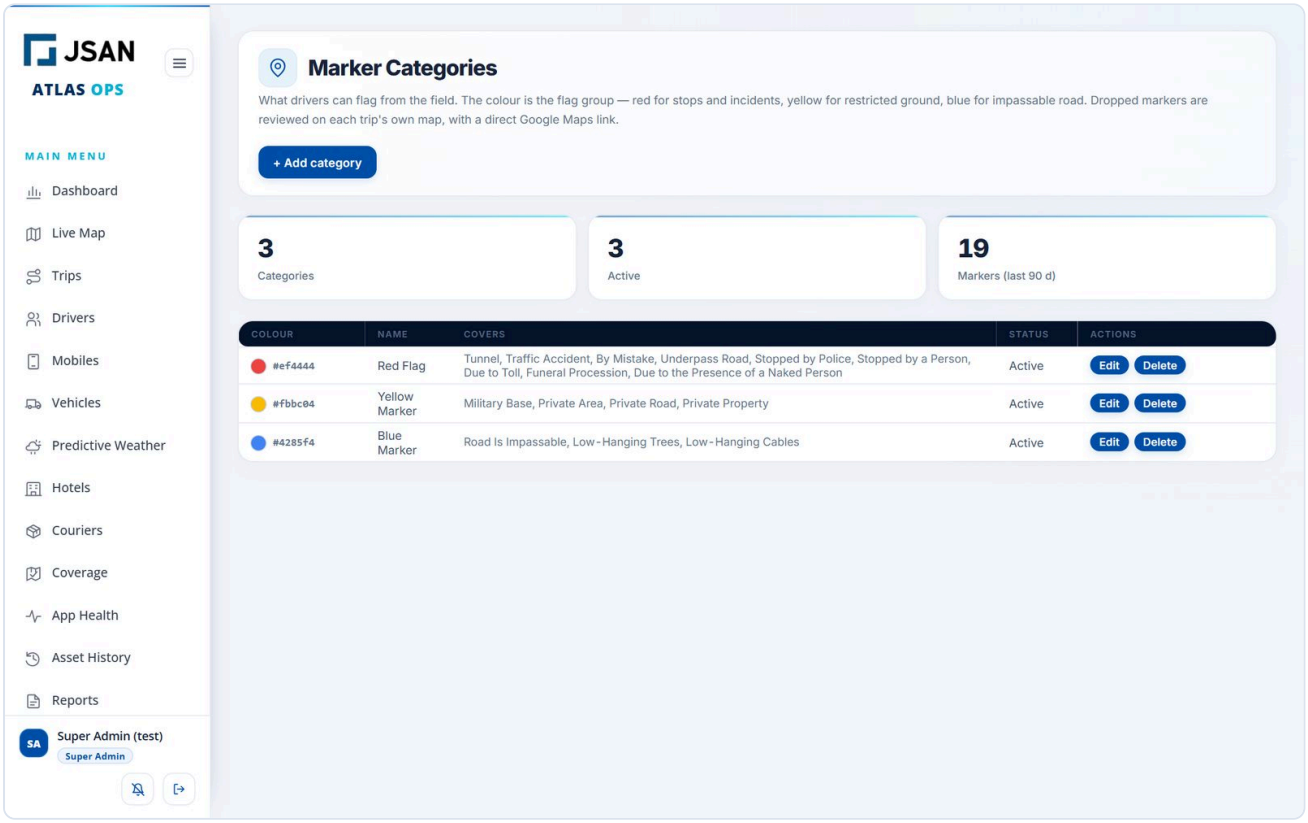
Mobiles — the phones issued to drivers (IMEI, model, number).



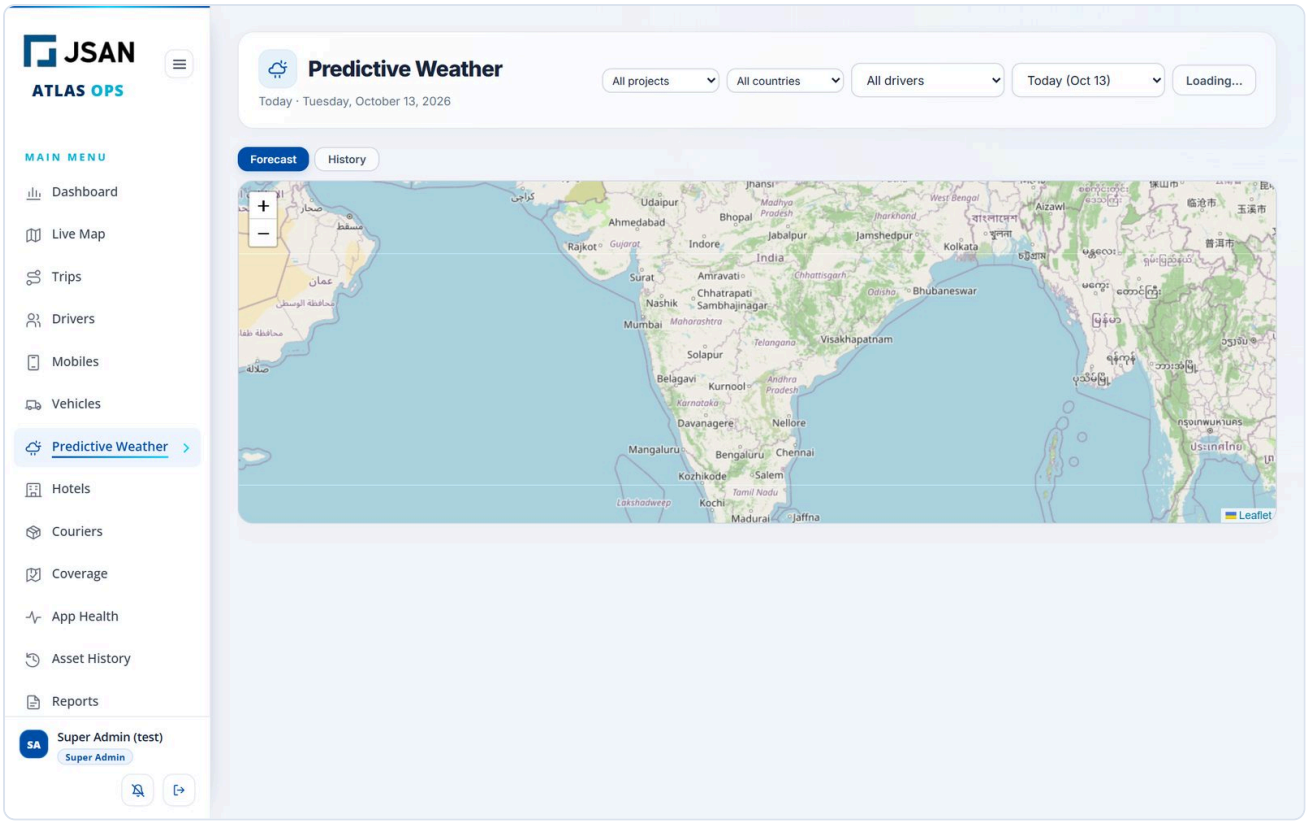
Projects — customer programmes, coverage scope and the per-project trip stop timeout.



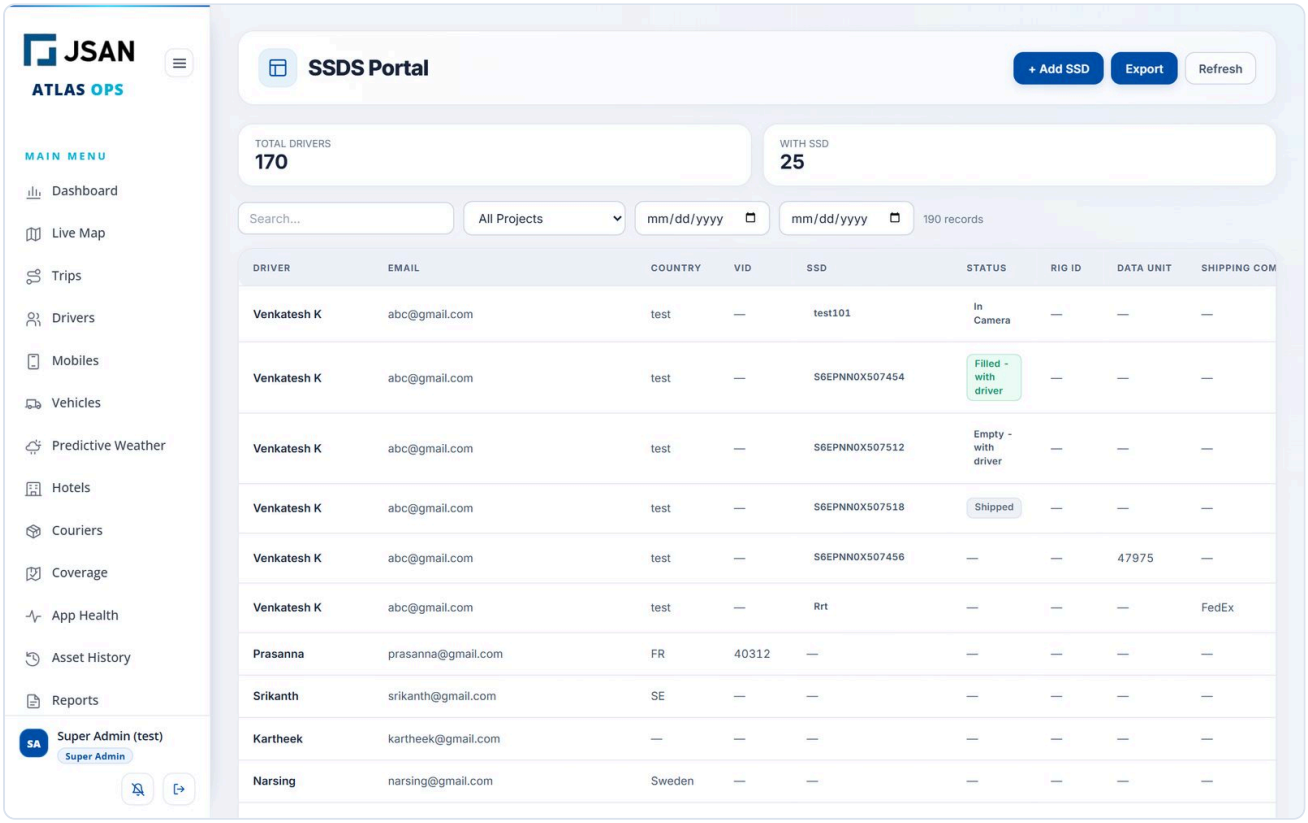
App Health — each driver's app status: last sign-in, last heartbeat, online / offline / warnings.



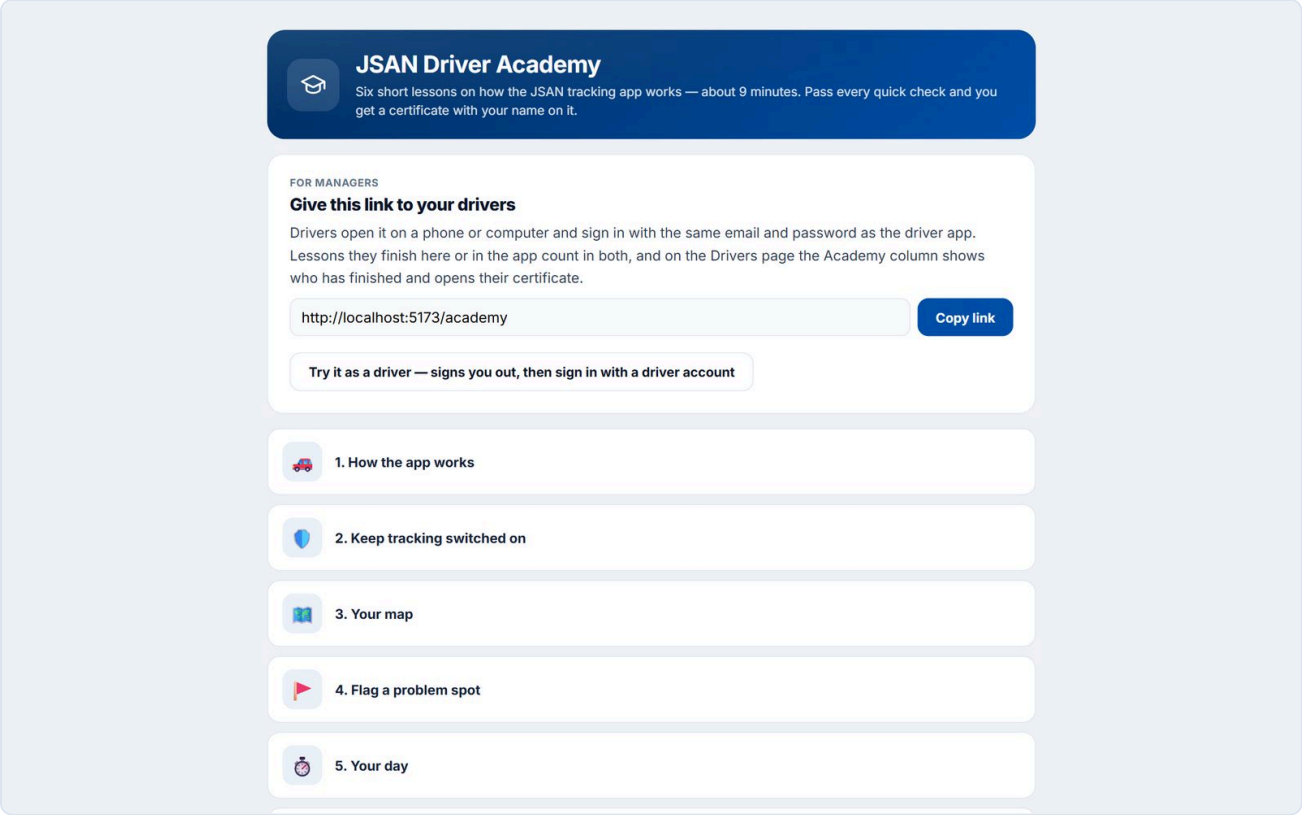
Marker categories — what drivers can flag (red: stops and incidents; yellow: restricted roads; blue: impassable).



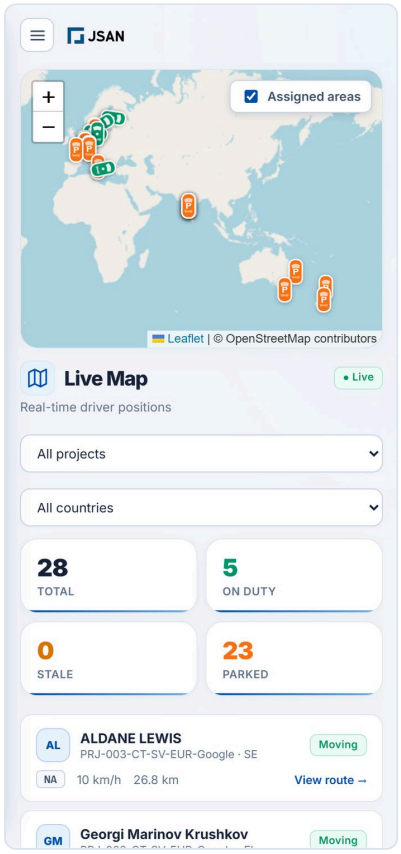
Predictive Weather — forecast along where crews are driving (Open-Meteo).



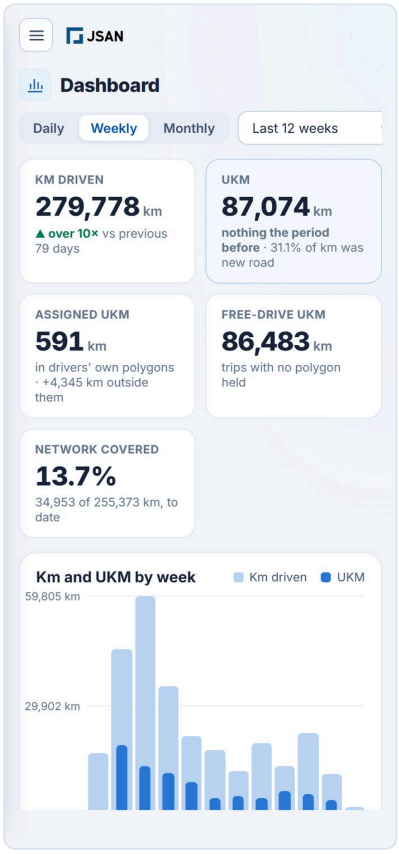
SSDS Portal — SSD card tracking per driver (cards filled, shipped, received), with timesheets and daily status reports alongside.



Driver Academy (web) — the shareable course link for drivers; six lessons, server-graded checks, certificate on completion.



Panel on a phone — Live Map; the panel installs as an app (PWA) and receives driver-offline push alerts.



Dashboard on a phone.

The Android driver app's own screens are not captured here; they can only be screenshotted from a phone.

12 External services

Service	Used for	Notes
Valhalla	Snapping trips to roads	Community server (`valhalla1.openstreetmap.de`); a gap-fill flag marks stretches it had to infer.
Open-Meteo	Weather forecast and history	Free, no key.
Nominatim	Place names for positions	OpenStreetMap geocoder.
Overpass API	Nearby places from OpenStreetMap	Main and fallback endpoints.
Serper	Courier office search	API key in env.
RapidAPI (Booking.com)	Hotel search near crews	Metered key in env.
Google Gemini	OCR of SSD card and shipping labels	Called from the browser; its key ships in the panel build, so keep it restricted.
Web Push (VAPID)	Driver-offline alerts to installed panels	Keys in env.
Tigris (S3)	SSD card photos	S3-compatible bucket.
OpenFreeMap / OSM tiles	Base maps in panel and app	Loaded directly by the browser / phone.
Expo EAS	Android builds and update channels	preview and production profiles.

13 Testing

- **41 backend test suites** (`npm test` in `backend/`): each starts an in-memory MongoDB (`mongodb-memory-server`) and calls the real Express app through `supertest` — ingest, trips, UKM, link coverage, area completion, splits, resets, regions, imports, road files, roles, analytics, Academy and more.
- **Admin panel:** TypeScript type-check (`tsc --noEmit`) and Vite production build.
- **Driver app:** the trip-start rule (StartGate) has a JavaScript mirror with a fuzz simulation; new builds are tried on one or two phones before being marked required.

14 Repository layout

Path	Contents
backend/src/routes, controllers, services, models	The API, in layers
backend/src/realtime/io.js	Socket.IO server
backend/src/seed/	One-off scripts: backfills, seeding, `makeSuperadmin`
backend/test/	The 41 test suites
admin-panel/src/pages, components, lib	Panel screens, shared UI, API / auth / socket clients
JSAN-AUTO-TRACKING/app	Driver app screens (Expo Router)
JSAN-AUTO-TRACKING/modules	Native Kotlin tracking engine
core-tracking-code/	Tracking reference for porting to other apps
docs/, README_GLOBAL_UKM_...md	Design specs (UKM, assigned coverage, custody, exports) and this document